Transformer

Language Models

Course Contents

Deep Learning
Transformer
LLMs
Context Engineering
Finetuning
Agents
The Bigger Picture

This Part: Transformer

- tokenizer
- static embeddings
- contextual embeddings
- text generation
- transformer types

Tokenization

tokenization: *breaking text in chunks*

- word tokens: different forms, spellings, etc → undefined and vast vocabulary
- character tokens: not enough semantic content (longer sequences)
- popular compromise: byte-pair encoding

Byte-Pair Encoding

data compression method used for encoding text as sequence of tokens (sub-words)

- merging token pairs (starting with characters) with maximum frequency
- continue merging until defined fixed vocabulary size (hyperparameter) is reached

→common words encoded as single token

→rare words encoded as sequence of tokens (representing word parts)

alternative: direct operation on bytes (e.g., ByT5)

aaabdaaabac

ZabdZabac
Z=aa

ZYdZYac
Y=ab
Z=aa

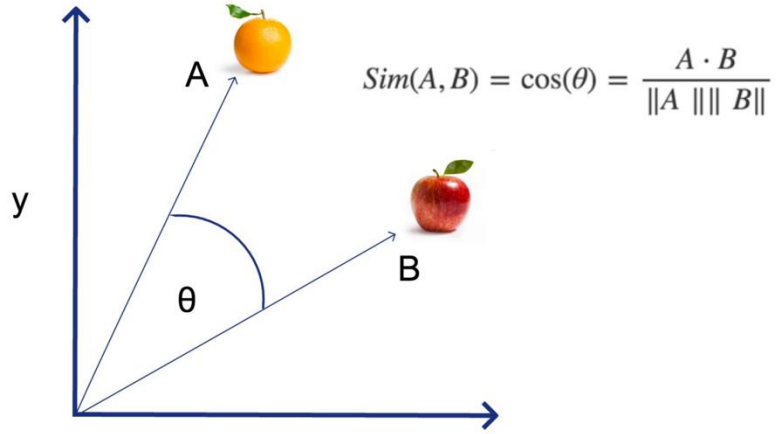
XdXac
X=ZY
Y=ab
Z=aa

example from wikipedia

Embedding Layers

representation of entities by vectors

→ semantic similarity



most famous application: word embeddings

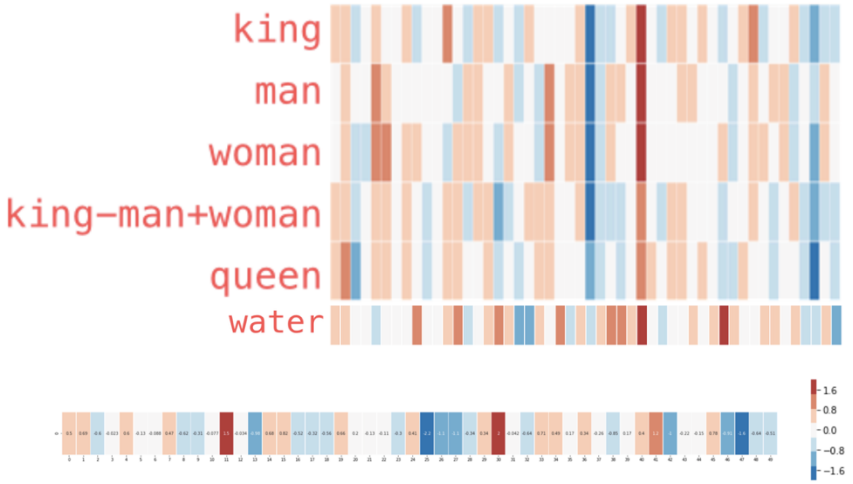
→ associations (natural language processing)

but general concept: embeddings of (categorical) features (e.g., products in recommendation engines)

learned via co-occurrence (e.g., word2vec)

but also direction of difference vectors interesting (analogies):

king − man + woman ~= queen



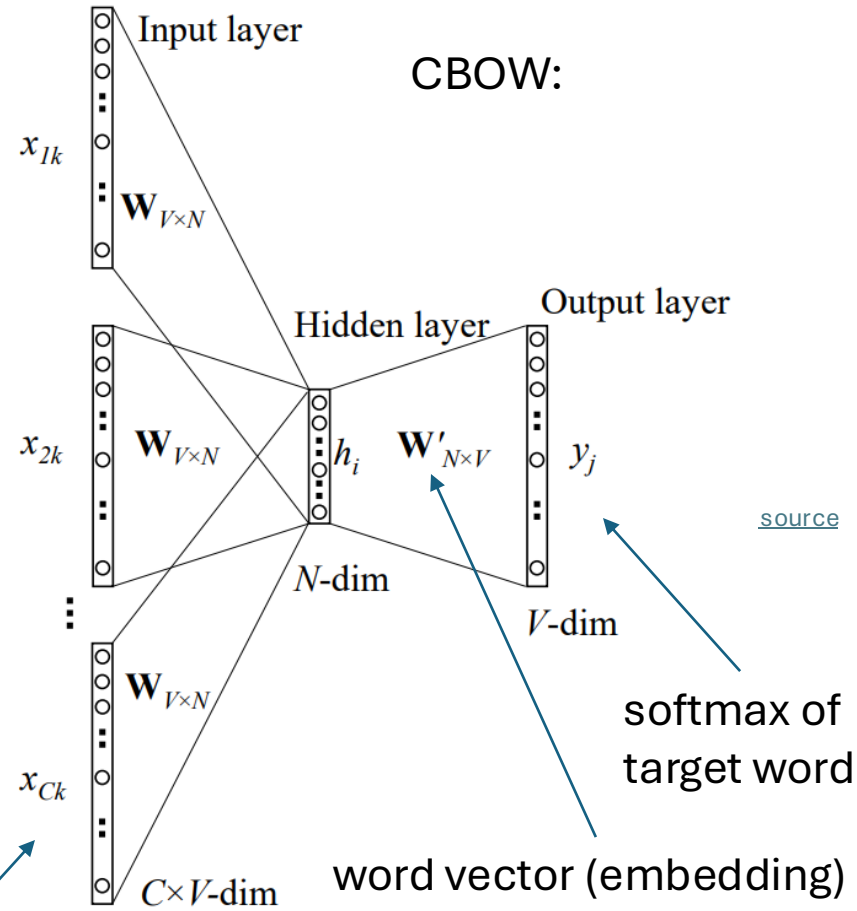
source

Some Thoughts on Word Embeddings

can be implemented as

- neural network with single hidden layer (linear activation)
- using, e.g., bag-of-words approach (predict masked word from its surroundings)

→ not context-aware



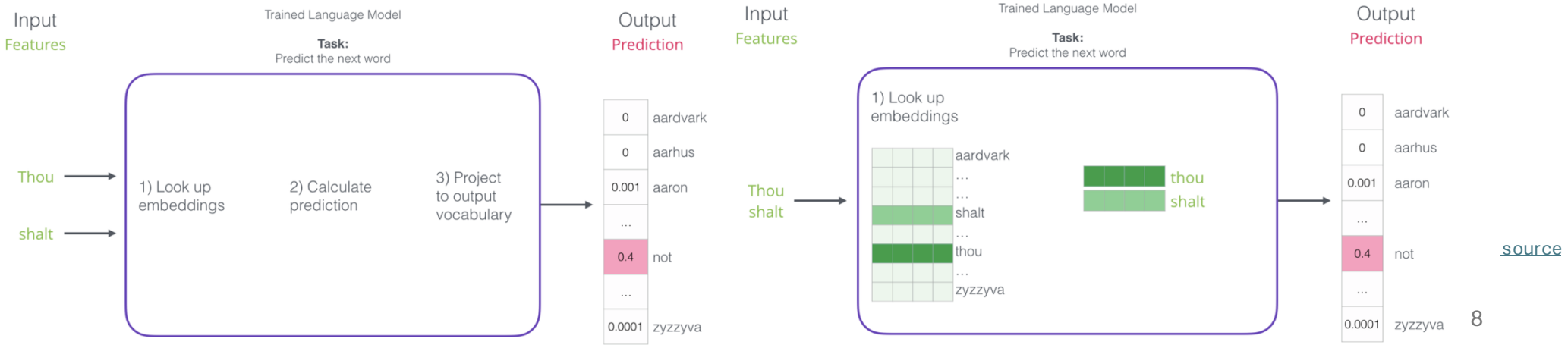
Word Embeddings as Part of Language Model

language models contain embedding matrix as part of learned parameters

typically several hundred dimensions for word vectors (to be compared with vocabulary sizes of many thousands)

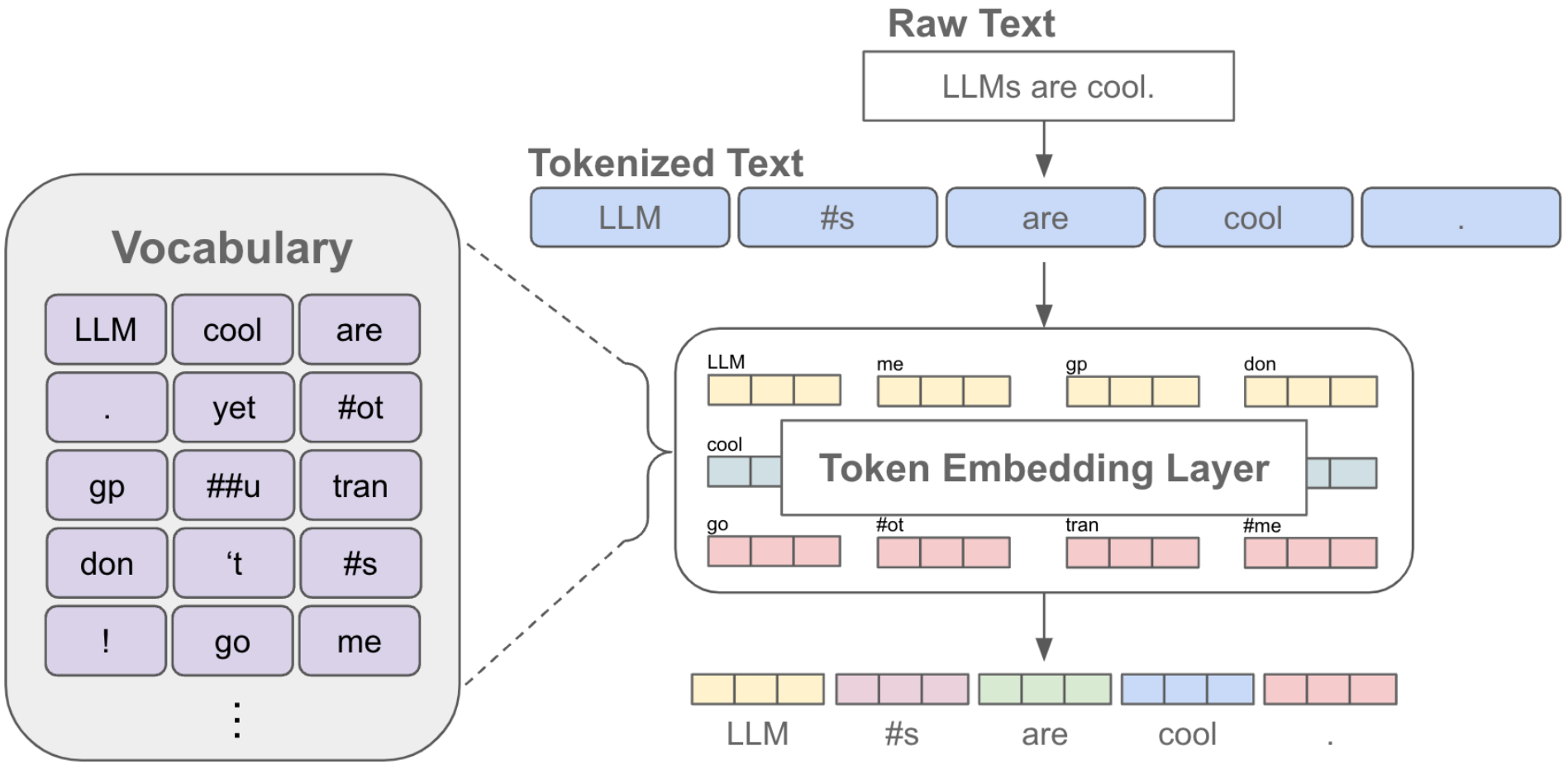
(can also be extracted and subsequently used as pretrained embeddings for other task)

next-word prediction:



source

Tokenization & Embeddings



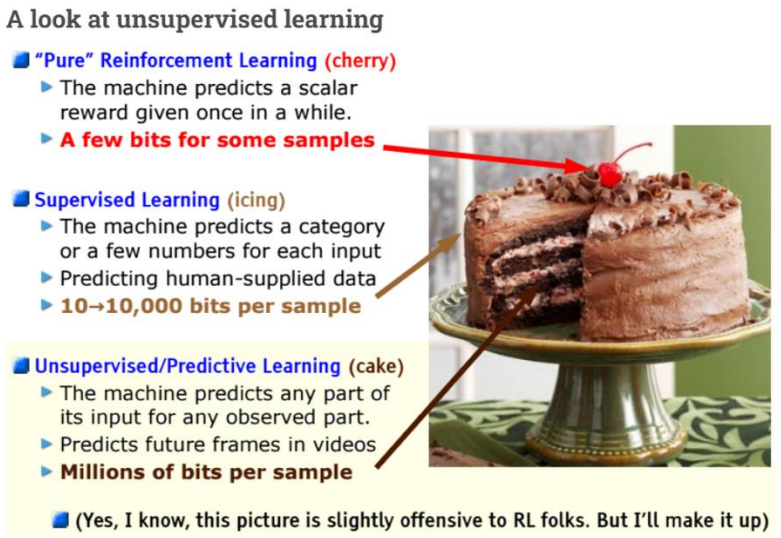
source

Self-Supervised Learning

ML needs lots of training data

unsupervised learning (learning by observation)

no target information → kind of "vague" pattern recognition (but plenty of data)



source

can be cast as **self-supervised learning**:

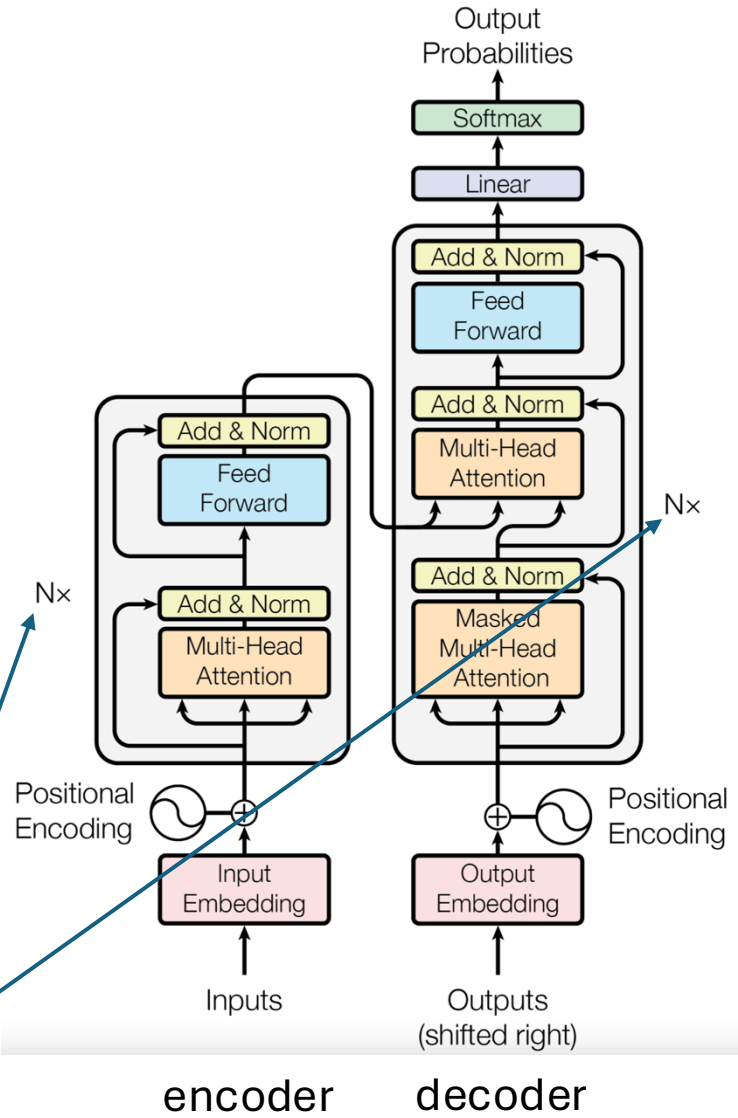
- input-output mapping like supervised learning
- but generating labels itself from inputs

```
The Lord of the Rings is considered one of the greatest
The Lord of the Rings is considered one of the greatest fantasy
                        ⋮
```

now run this across the entire internet …

Context Awareness: Transformer

original transformer:
sequence-to-sequence model
(e.g., for machine translation)



[Attention Is All You Need]:

completely replaced recurrent neural networks (RNN)

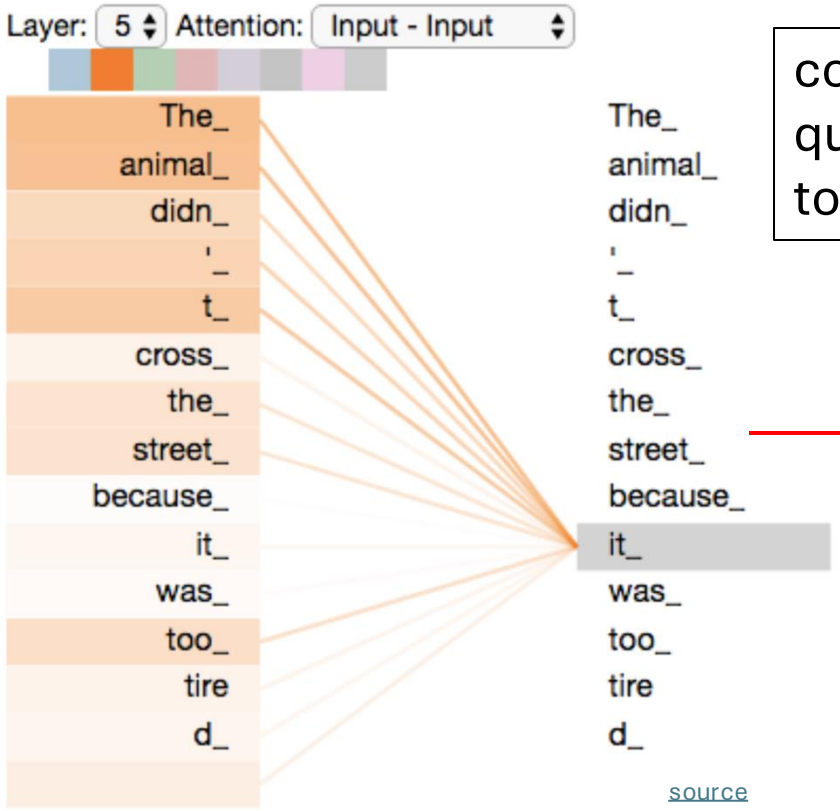
much more parallelization → bigger models

better long-range dependencies thanks to shorter path lengths in network (less sequential operations)

encoder/decoder stacks (depth):
output fed as input to next one

Self-Attention Mechanism

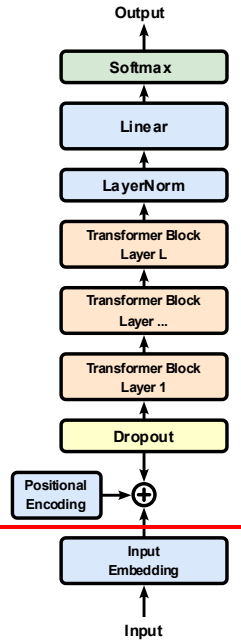
evaluating other input tokens in terms of relevance for encoding of given token

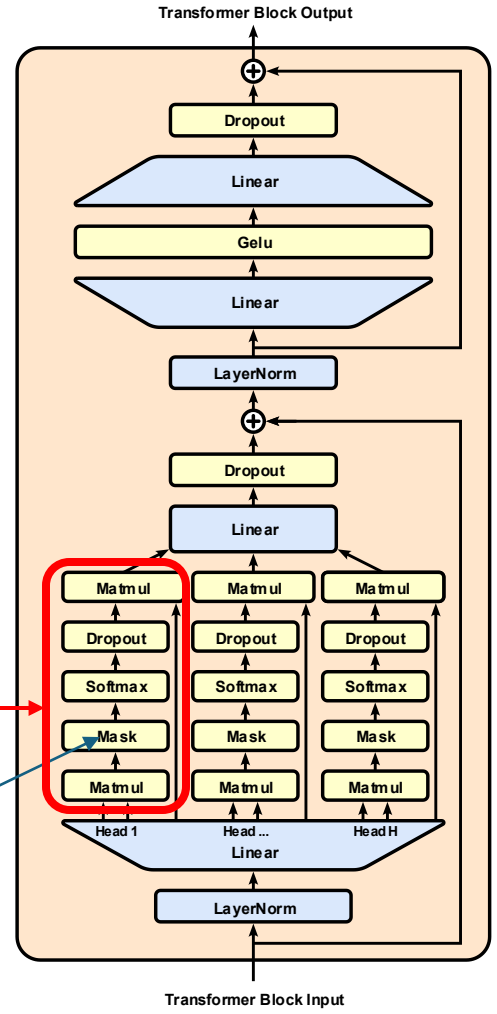


source

computational complexity quadratic in length of input (each token attends to each other token)

GPT (decoder-only):





only allow to attend to earlier positions in sequence (masking future positions)

Scaled Dot-Product Attention

softmax not scale invariant (largest component dominates for large scale) and dot product larger for more vector dimensions

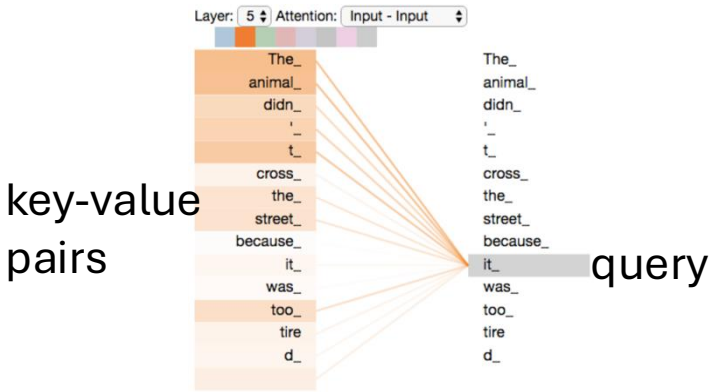
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Scaled Dot-Product Attention

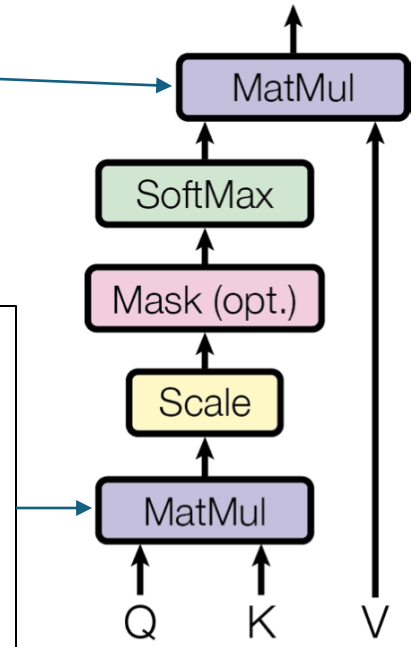
3 matrices created from input embeddings by multiplication with 3 different weight matrices

- query Q
- key K
- value V

filtering: multiplication of attention probabilities with corresponding key-word values



scoring each of the key words (context) with respect to current query word: correlation between inputs (not independent as in conventional neural networks)



source

Example: Dimensions in BERT

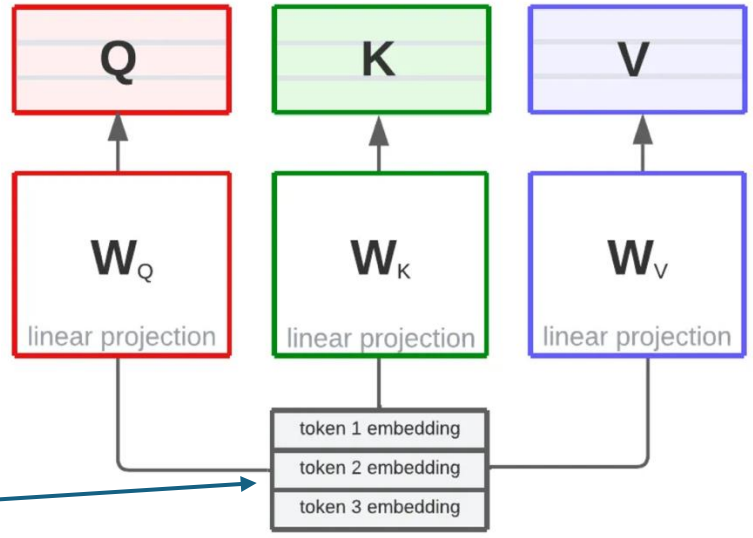
$$W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

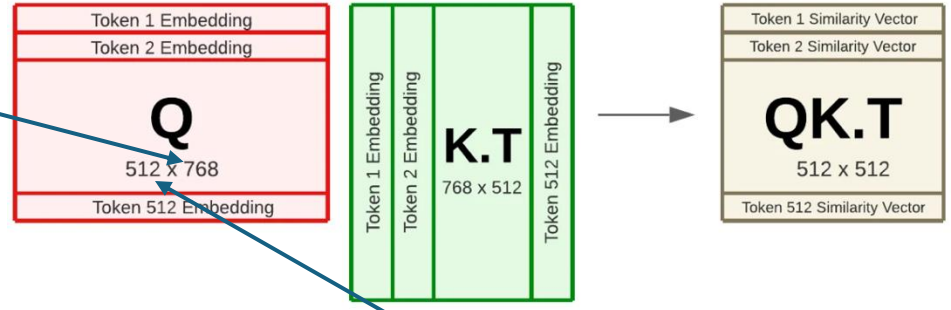
often $d_k = d_v = d_{\text{model}}$
(but not necessarily)

linear projections

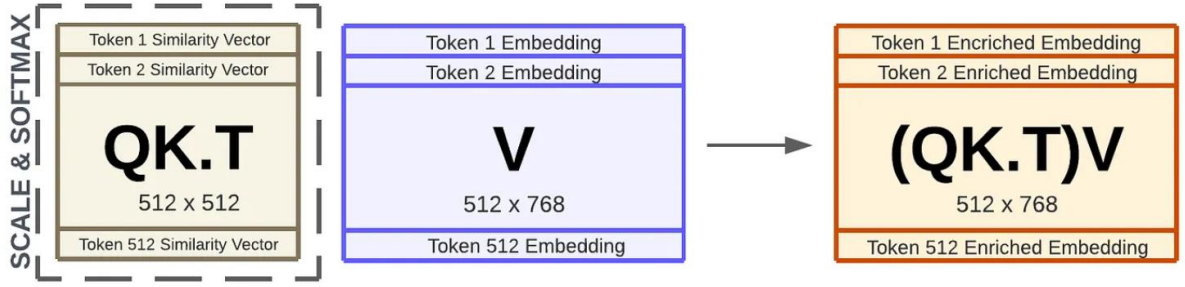
learned parameters

inputs



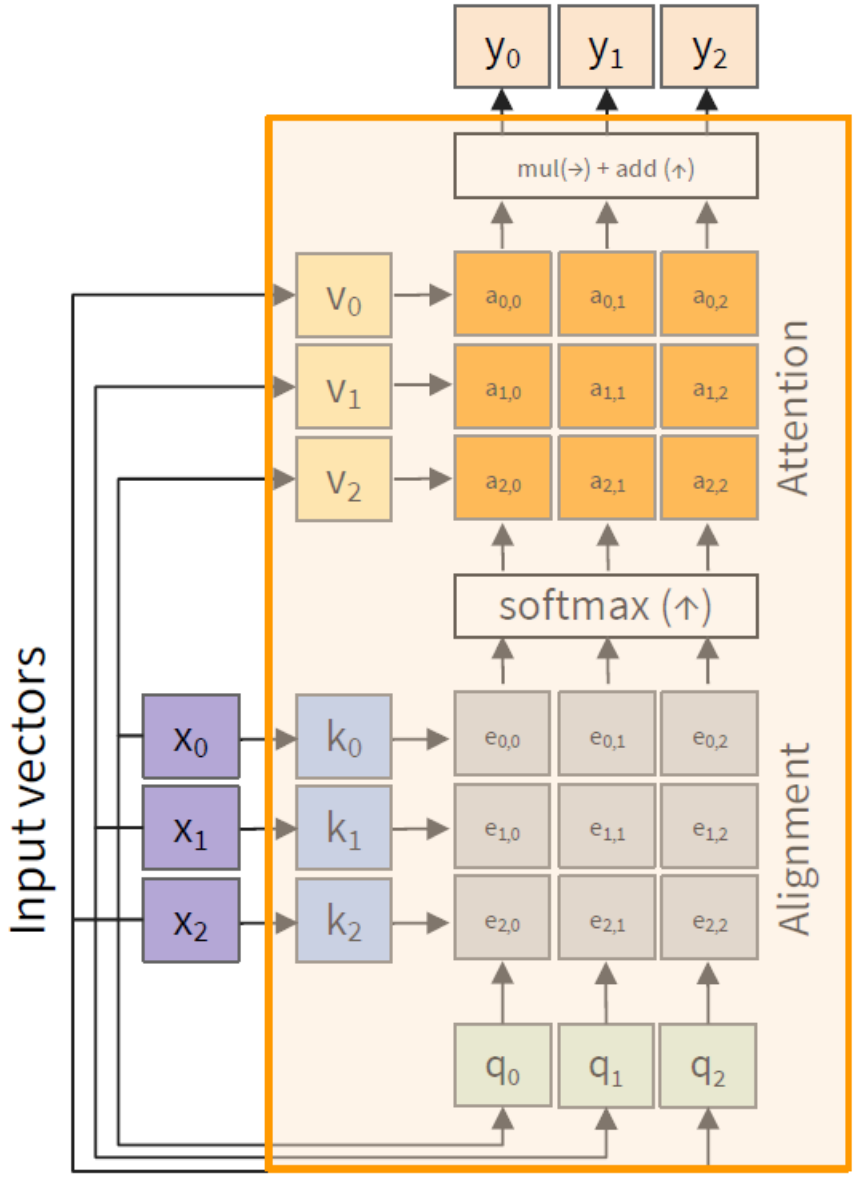


sequence length n
upper bound: context length of model
(design choice)



source

MatMul



Outputs:
context vectors: y (shape: D_v)

d_v

Operations:
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Query vectors: $q = xW_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

d_k

d_{model}: embedding size

Inputs:
Input vectors: x (shape: N x D)

weighted average: reflecting to what degree a token is paying attention to the other tokens in the sequence

Attention Mask

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

attention mask

causal masking:

dynamically mask future positions in sequence by setting to $-\infty$

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Padding Mask

goal: (technically) batch together sequences of different lengths

→ include fake tokens to pad shorter ones to the max length in the batch

then add a padding mask to let the model ignore these fake tokens:

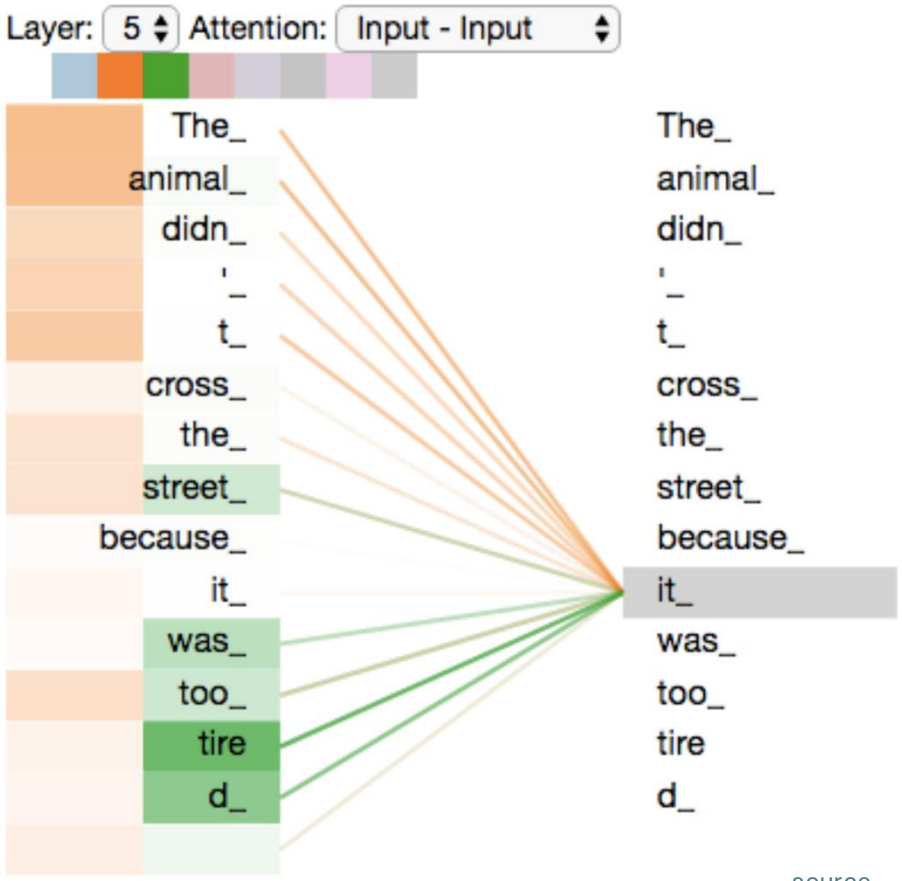
$$M = M_{\text{causal}} + M_{\text{padding}}$$

$$M_{\text{padding}}(i, j) = \begin{cases} 0 & \text{if token } j \text{ is valid} \\ -\infty & \text{if token } j \text{ is padding} \end{cases}$$

Multi-Head Attention

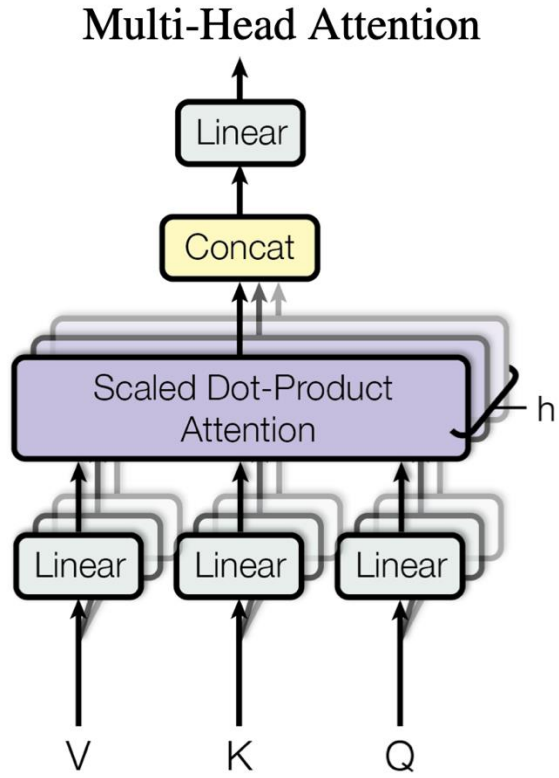
often $d_k = d_v = d_{\text{model}}/h$
and all h heads share the same d_k and d_v

multiple heads: several attention layers running in parallel



source

different heads can pay attention to different aspects of input (multiple representation sub-spaces)



source

Positional Encoding

defines context length

attention permutation invariant → need for positional encoding to learn from order of sequence

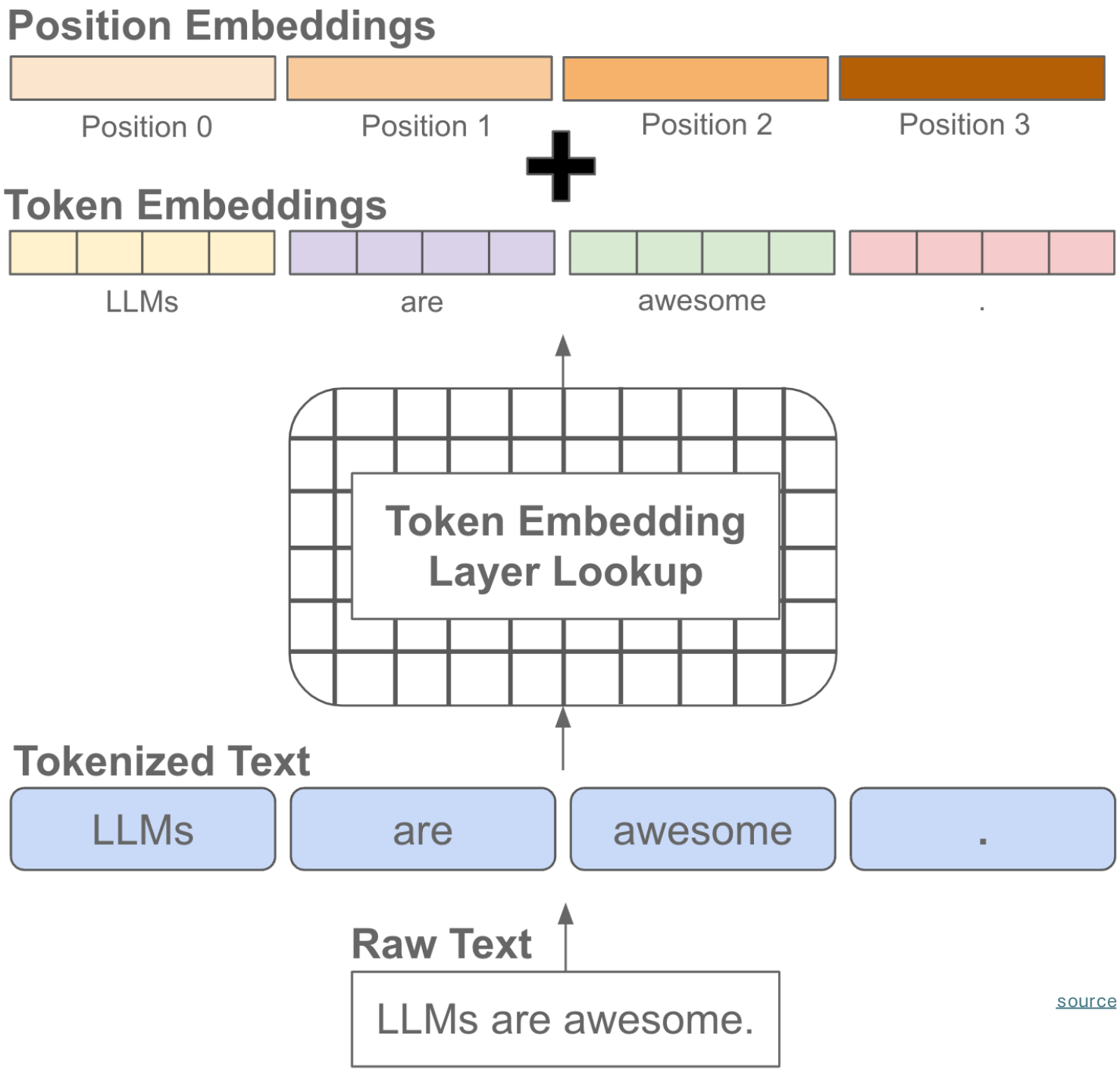
different options:

- for each absolute position (from 1 to maximum sequence length), add a learned vector (of dimension d_{model}) to input embeddings → each positional embedding independent of others
- add fixed sinusoidal functions for each position and dimension i

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \qquad PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

- rotate input embeddings by multiples (position) of a small angle (RoPE) → captures both relative and absolute positions

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W^{(11)}_{\{q,k\}} & W^{(12)}_{\{q,k\}} \\ W^{(21)}_{\{q,k\}} & W^{(22)}_{\{q,k\}} \end{pmatrix} \begin{pmatrix} x^{(1)}_m \\ x^{(2)}_m \end{pmatrix}$$


Position Embeddings
Position 0
Position 1
Position 2
Position 3
Token Embeddings
LLMs
are
awesome
.
Token Embedding Layer Lookup
Tokenized Text
LLMs
are
awesome
.
Raw Text
LLMs are awesome.

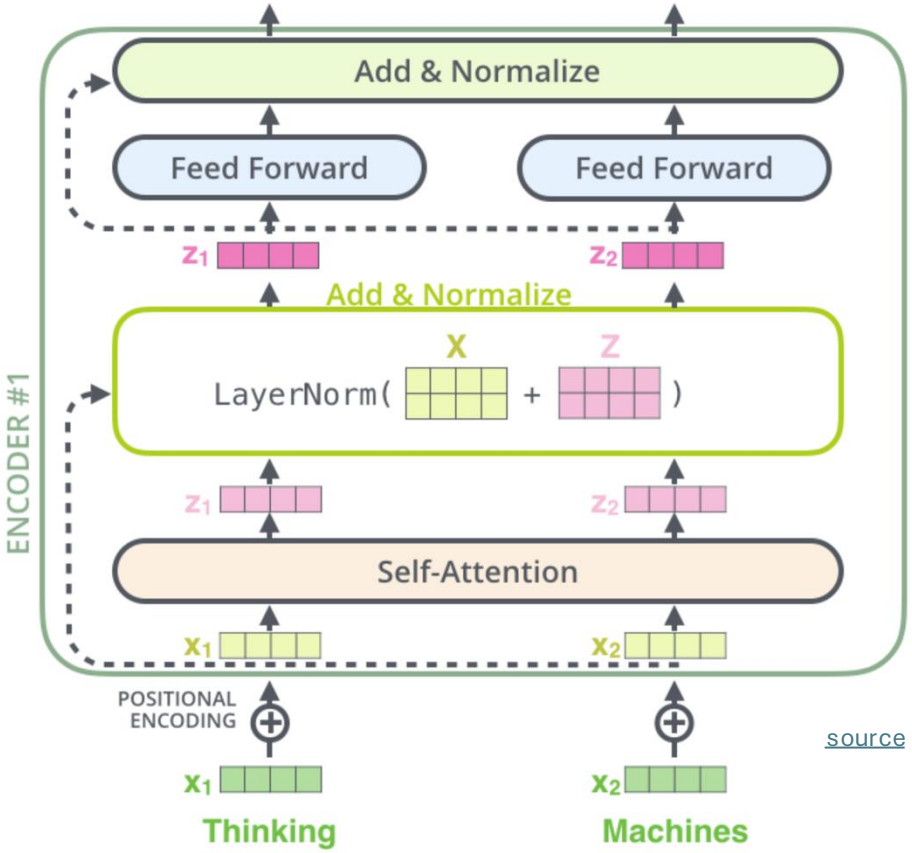
source

Skip Connections and Layer Normalization

skip connections improve robustness by

- preserving original input (attention layers as filters) as well as gradients (mitigate vanishing-gradient problem)
- easier learning of identity functions (useful for disregarding modules that do not improve model performance)

layer normalization improves stability, convergence, and generalization by normalizing activations per input token

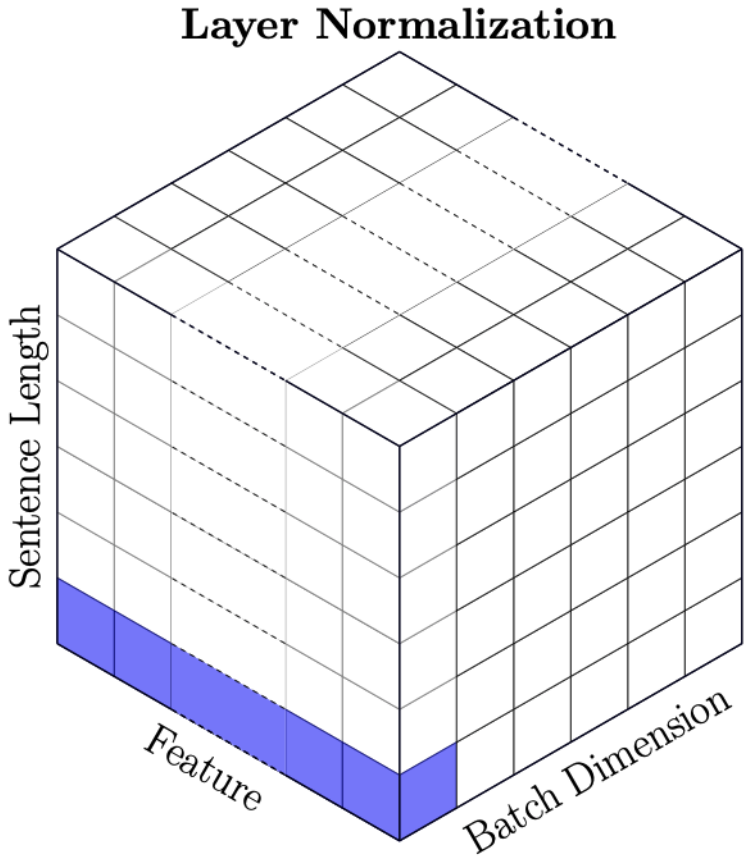


source

Batch vs Layer Norm

batch norm typically in computer vision

layer norm in NLP (variable-sized inputs)



Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

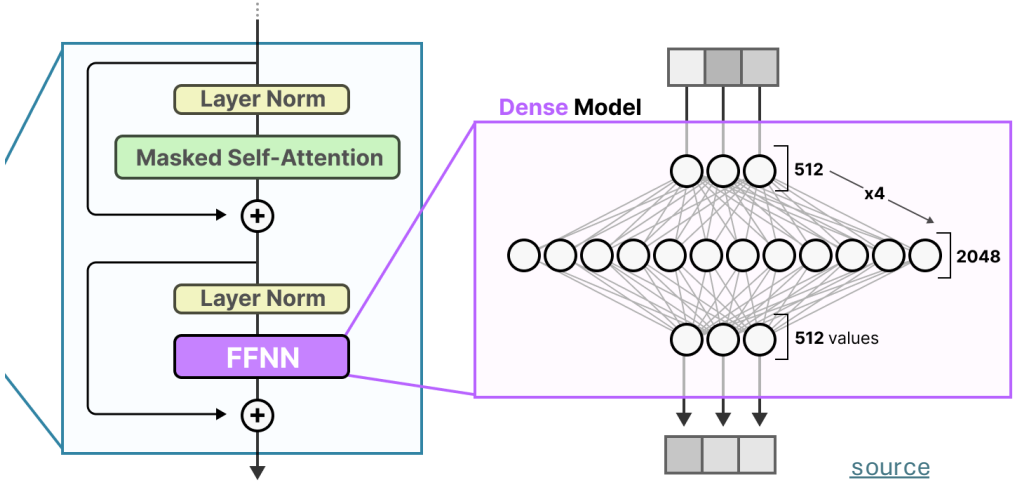
source

Role of Feed-Forward Neural Networks (FFNN)

position-wise FFNNs:

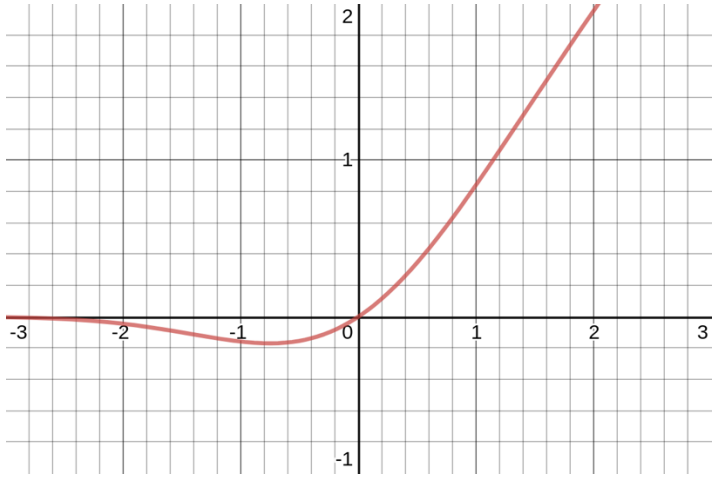
following each attention layer, apply an identical FFNN (with two layers) independently to each embedding vector

(correspond to bulk of overall parameters)



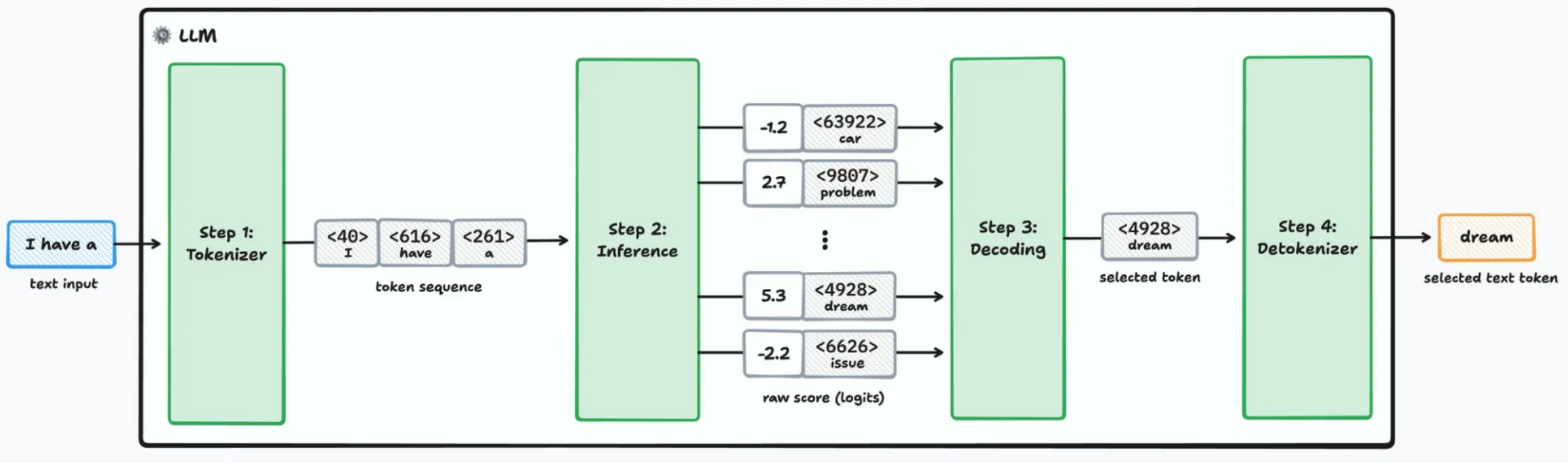
can be interpreted as compute after connect (attention)

attention is just weighted averaging
→ need for non-linearities (often GeLU activations) to capture more complex patterns



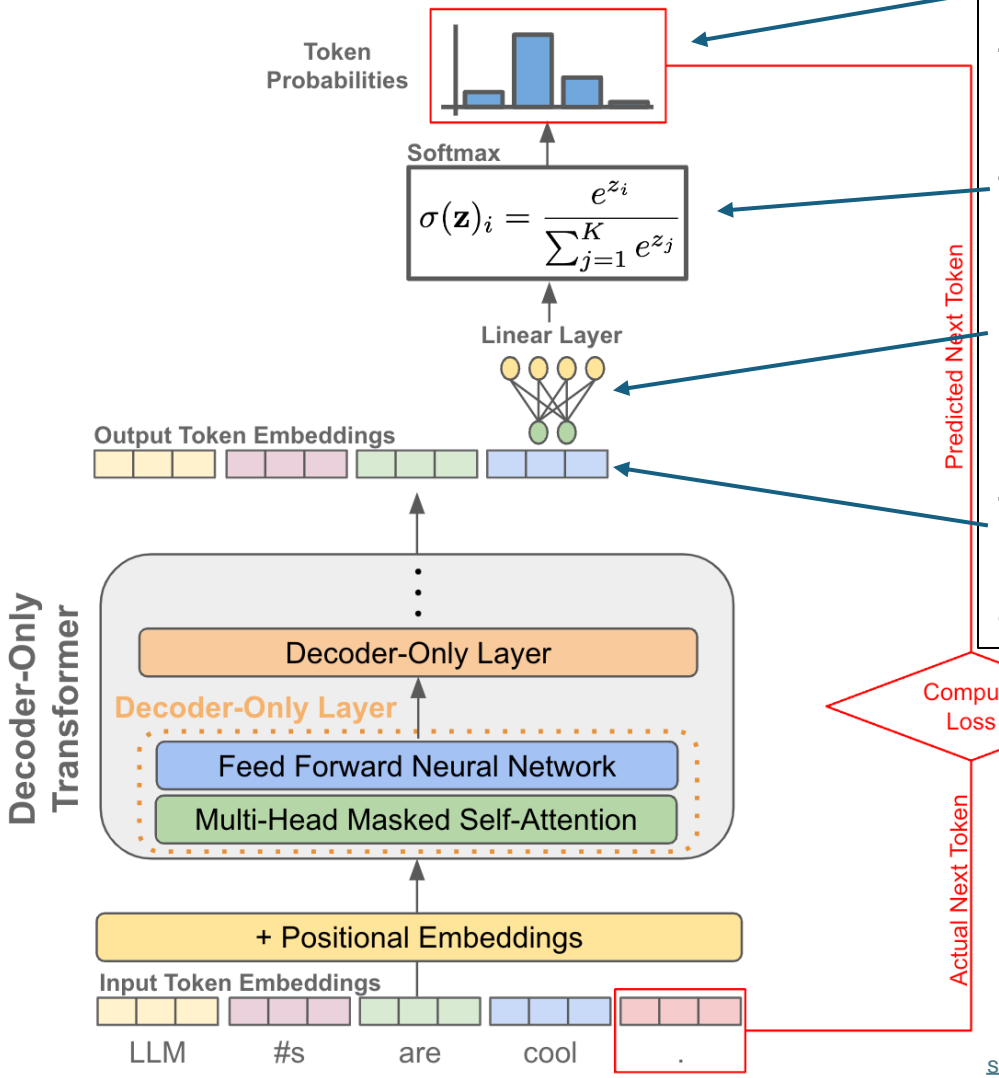
Next-Token Prediction

forward pass:



source

From Embeddings to Probabilities



source

next-token probabilities for each token in vocabulary

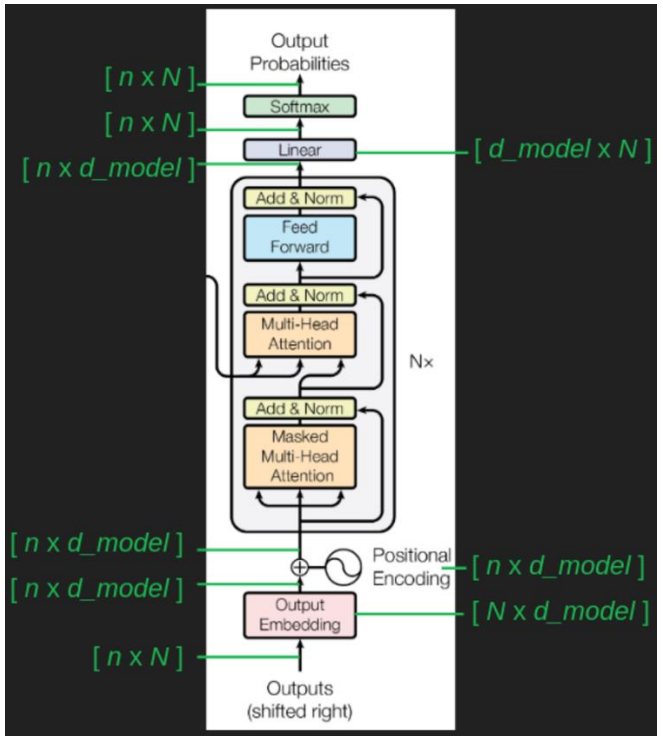
transformation to probabilities

de-embedding to vocabulary: projection to vector of logits

for projection, only use contextual embedding of last token (still need all contextual embeddings due to matrix multiplication in self-attention)

(masked token for encoder-only transformers)

n: sequence length
N: vocabulary size
d_model: embedding size



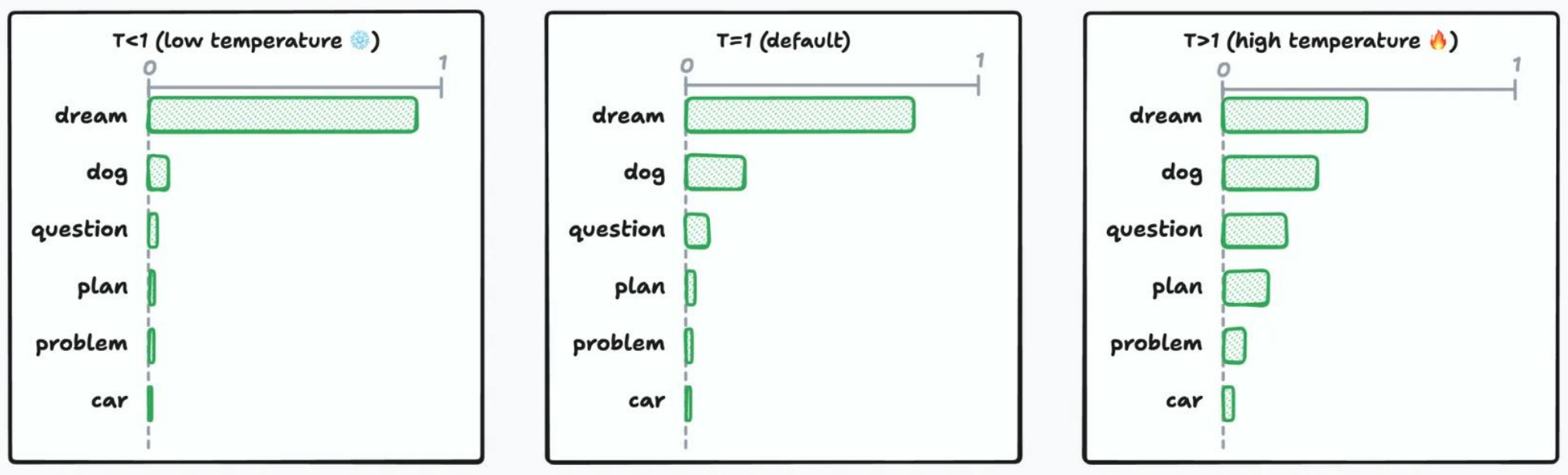
source

Token Selection at Inference

typically, pick according to probabilities

degree of randomness can be controlled by temperature parameter T

$$\text{softmax}(z_i) = \frac{e^{z_i/T}}{\sum_{j=1}^{n} e^{z_j/T}}$$

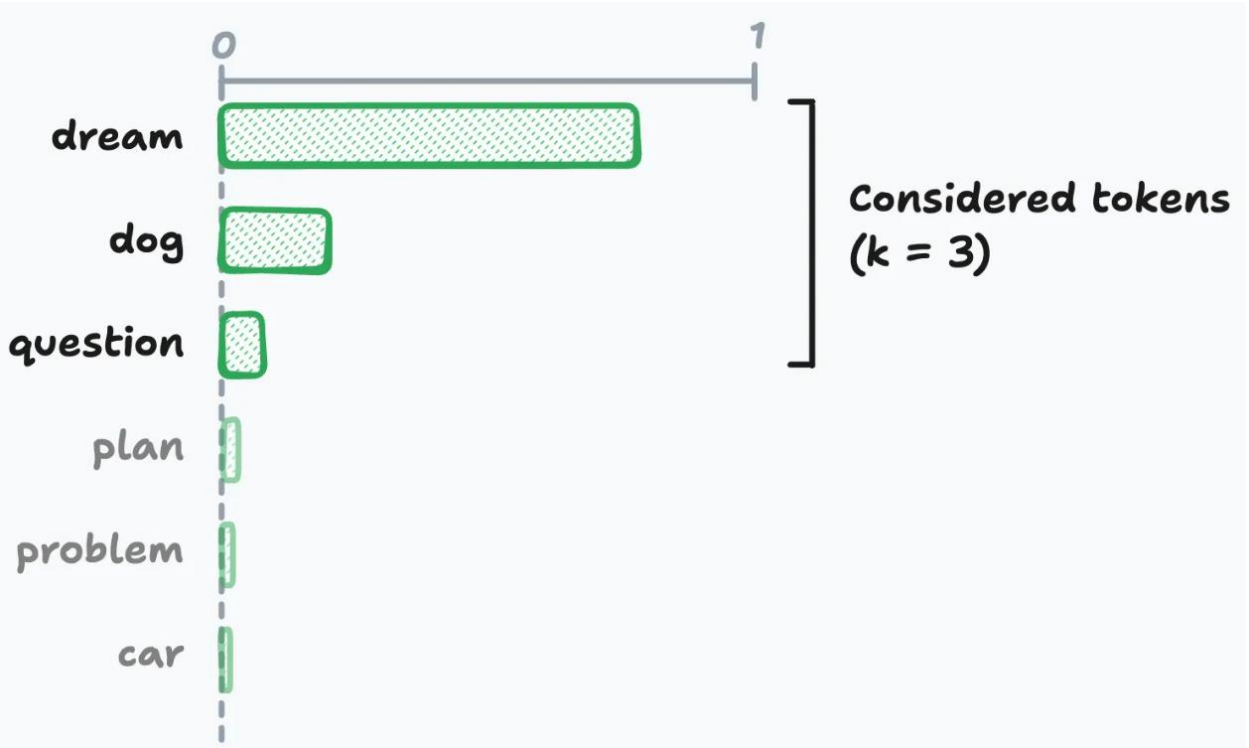


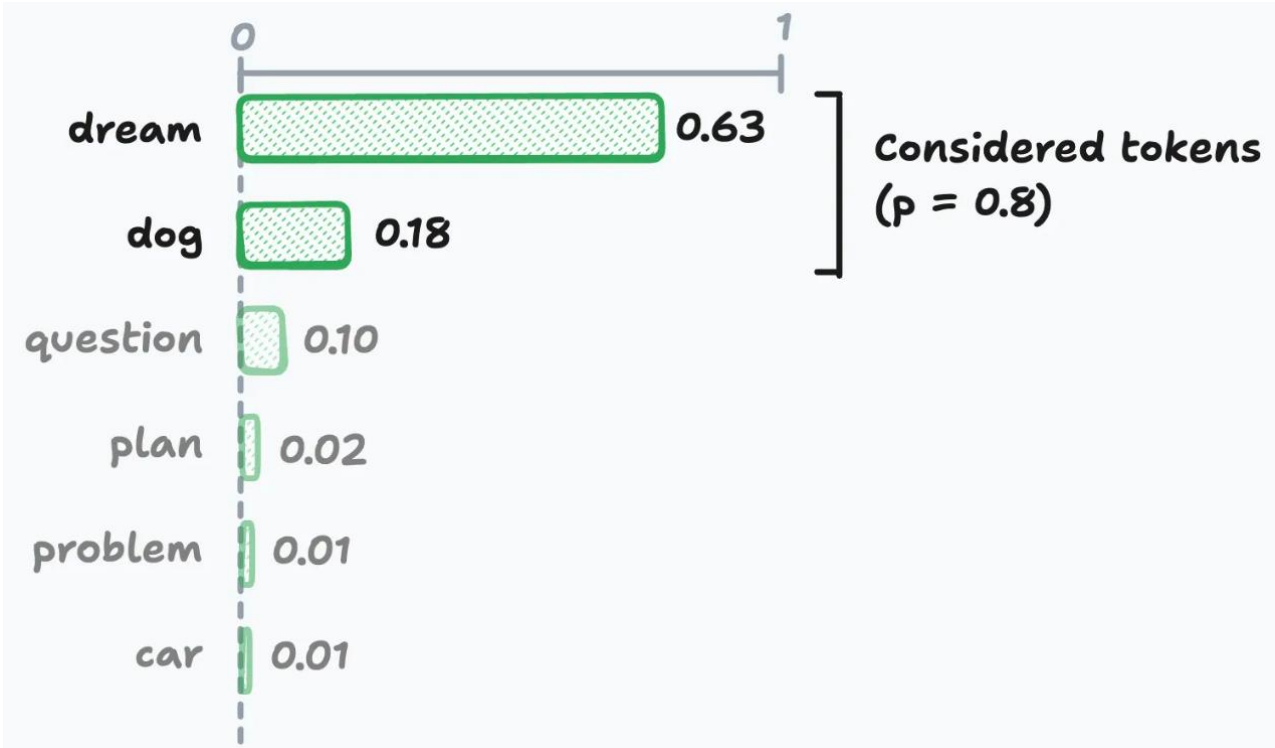
source

$T \to 0$ → greedy

creativity ;)

top-k & top-p Sampling



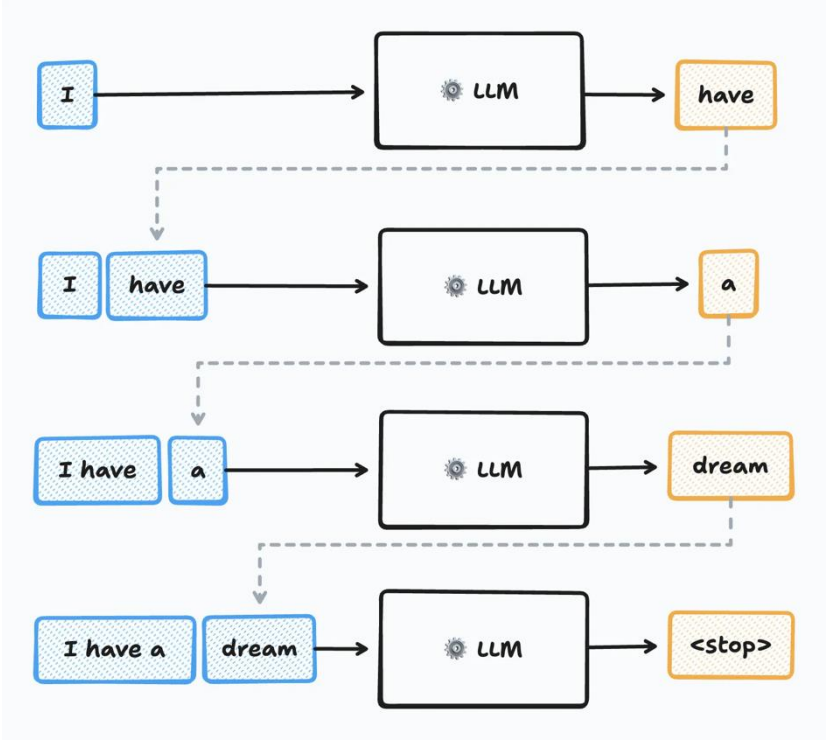


source

Sequence Completion

autoregressive:

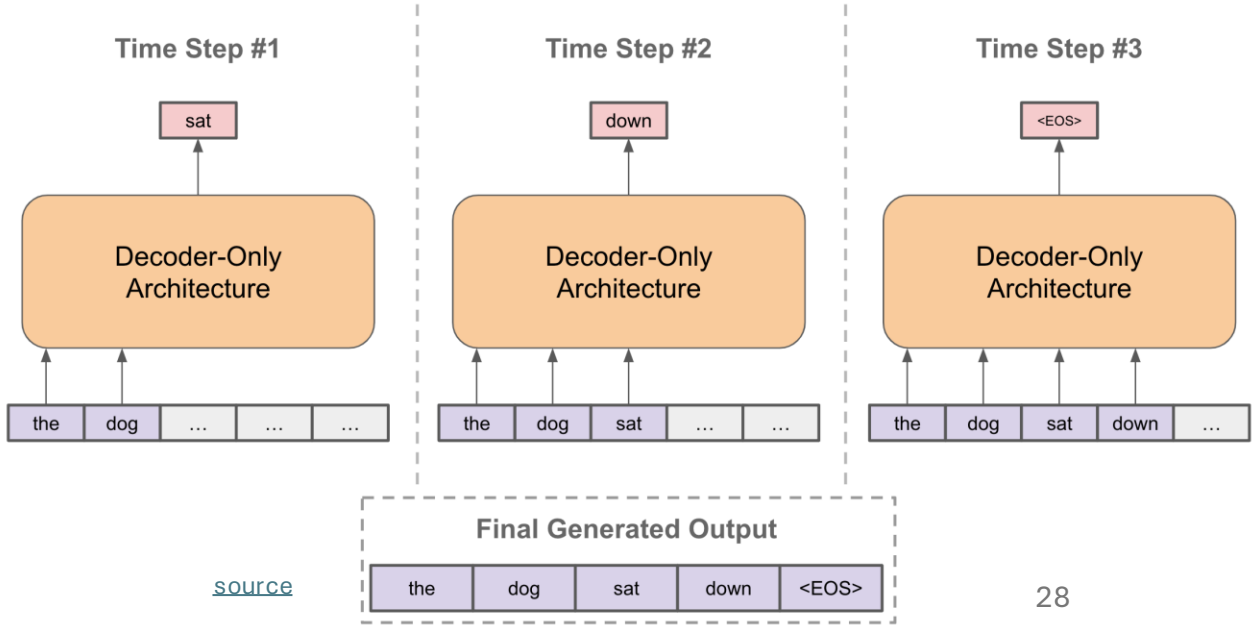
at each step, choose one output token, then add it to decoder input sequence for next step



source

prompt:

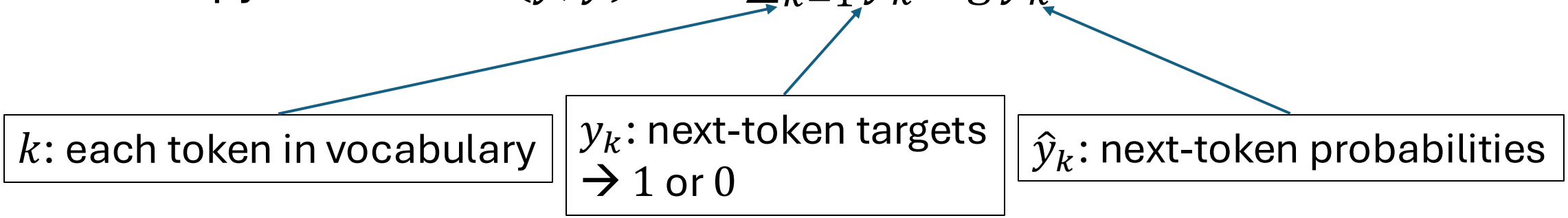
externally given initial sequence for running start and context on which to build rest of sequence



source

Training Loss

cross-entropy loss: $L(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$



during training, a next-token-prediction loss is calculated for every position in input sequence (not just the last one)
→ earlier positions have shorter effective contexts (masking of future positions)

reason: mirroring actual inference case of autoregressive text generation and arbitrary prompts

total loss = sum of individual losses

Teacher Forcing

in training, ground-truth previous tokens are used as inputs for next-token predictions from each position

(instead of own sampled predictions as in inference)

→ reduces variance and allows to compute an individual loss for every token in the sequence in parallel

KV Caching

during autoregressive generation, for each new token, key and value vectors (K and V) of all the previous tokens are needed when computing attention

but due to causal masking, K and V of previous tokens don‘t change from step to step (only the Q vector of the new token changes)

so instead of recomputing K and V of all all previous tokens at each step:

- store K and V for all tokens after its first computation
- for each new token, compute Q and its attention against the cached K and V

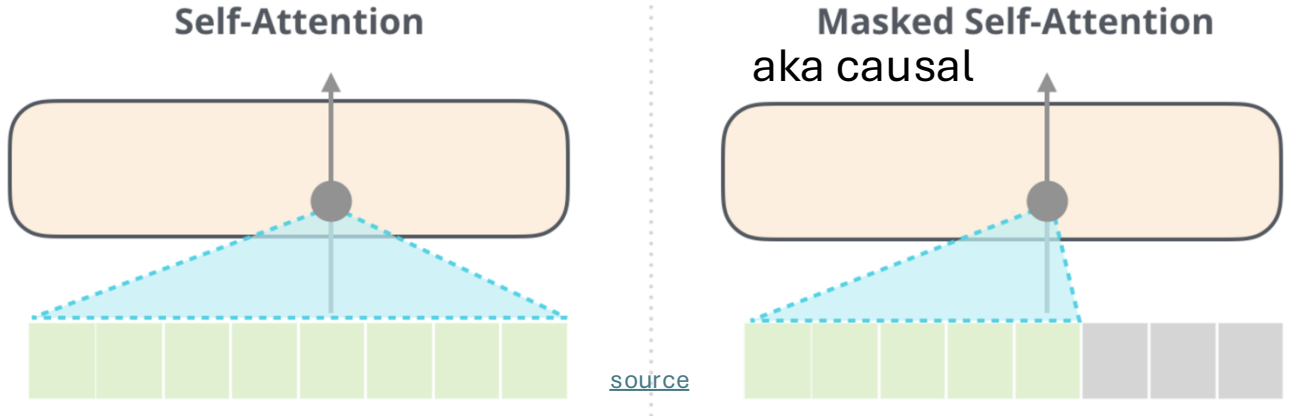
→ avoids redundant calculations

Transformer Types

encoder-only transformers:

- goal: language understanding
- training: masked-token prediction

prime example:
BERT



source

decoder-only transformers:

- goal: text generation
- training: next-token prediction

prime example:
GPT

encoder-decoder transformers:

- goal: sequence-to-sequence models
- training: next-token prediction

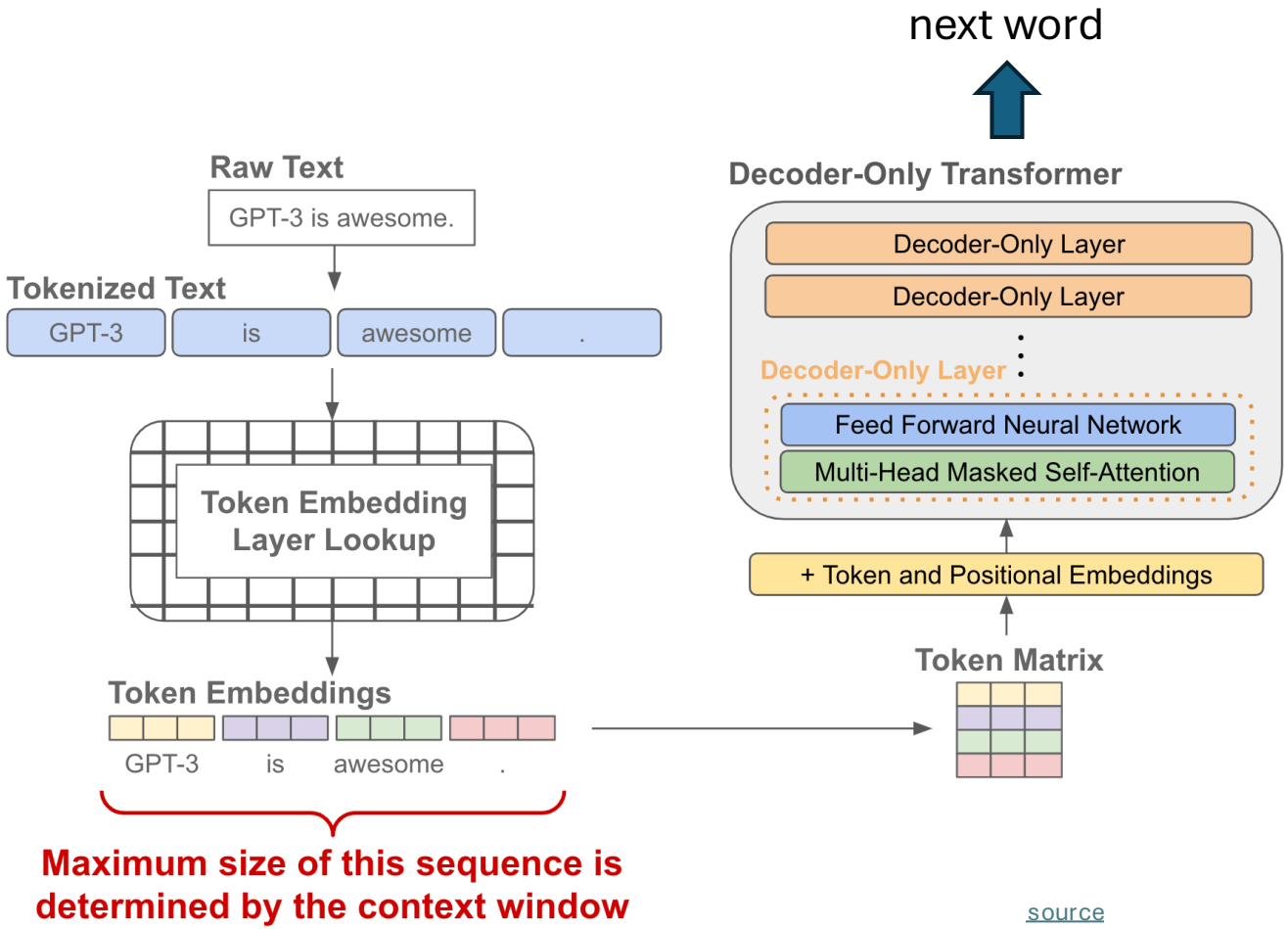
Decoder-Only = Autoregressive Models

today typically synonymous with LLM

stack of transformer decoders

→autoregressively generated sequence of tokens (directly usable as backbone of a chatbot)

training objective: next-token prediction from context



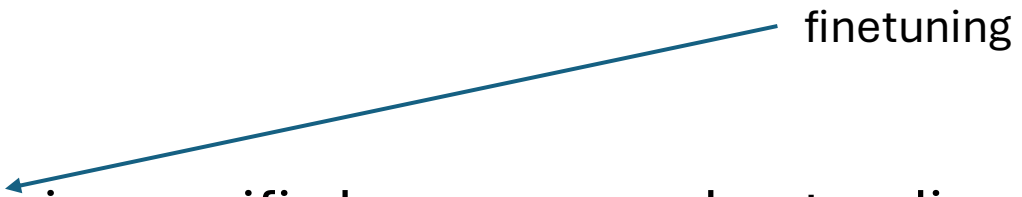
source

Encoder-Only = Bidirectional Models

training objective: masked tokens to be predicted from context

→ aka masked language models (MLM)

bidirectional: jointly conditioning on both left and right context



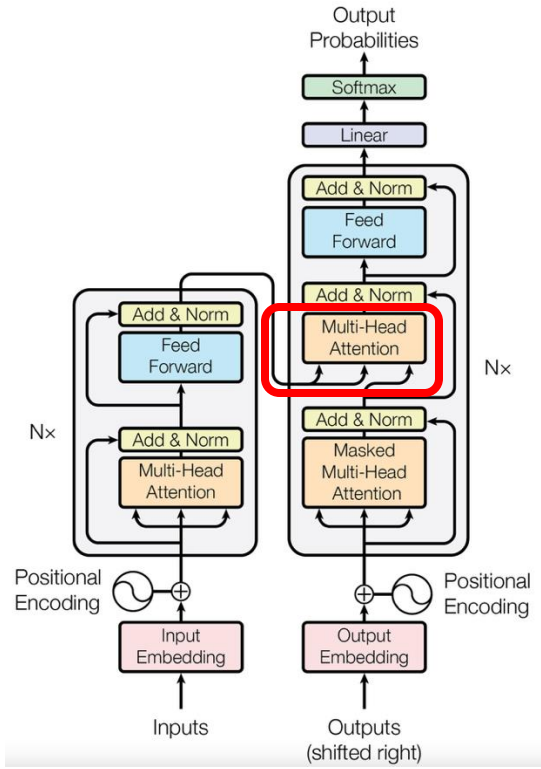
stack of transformer encoders

→ contextual embeddings (for later use in specific language-understanding tasks like text classification, sentiment analysis, named entity recognition)

still needs a decoder head for training purposes, using the hidden state of the masked token (dedicated [MASK] token) for projection to vector of logits

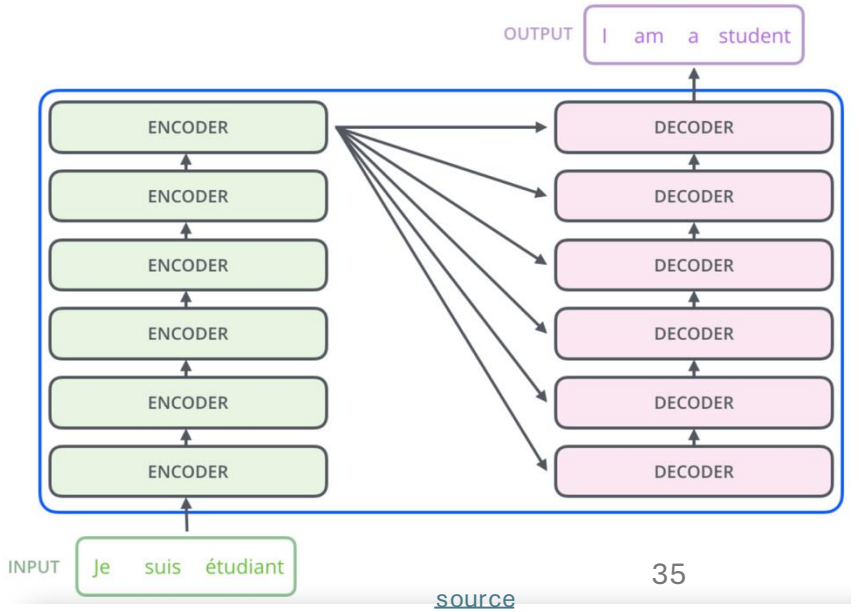
Encoder-Decoder Transformers

- use case: sequence-to-sequence models
- e.g., machine translation
- can also be done with decoder-only transformers, but encoder-decoder model sometimes beneficial
- examples: T5, FLAN



encoder-decoder attention (aka cross-attention):

- connection between encoder and decoder, helping decoder to focus on relevant parts of input sequence
- output of last encoder transformed into K and V, Q from each decoder's cross-attention layer



source

Next: LLMs

autoregressive text generation of decoder-only transformers allows prompting of models (can output text to arbitrary topics)

→ enables chatbots like ChatGPT

triggered hype about generative AI and started an LLM scaling race (the larger the model, the better the capabilities)

lightweight PyTorch re-implementation of GPT: minGPT
more powerful: nanoGPT, LitGPT